Done

Welcome Shashank Prakash from Database Architecture, Scale, and NoSQL with Elasticsearch

On ACID and BASE Architectures

The topic for this week is to compare, contrast, and explore the interplay between ACID and BASE style architectures. In this mini-paper (3-5 paragraphs) explain your take or views on these competing approaches to database architecture. Feel free to bring prior experience that you might have into the conversation. It is quite OK to take and support a position that is different from the position in this week's lectures. Let's see what we can learn from talking and listening to each other's perspectives. The main goal is to have a conversation.

Don't take off points for little mistakes. Feel free to make suggestions if there are small mistakes. Please keep your comments positive and useful. Provide comments that will help us all think about the question that is posed. Some of the points will be given by the instructors - it might take a while for the instructor points to appear.

Please Upload Your Submission:

Three to five paragraphs of your take on ACID versus BASE architectures, past, present, and/or future

Paragraph

Benefit ACID in BFSI

In my experience as a data scientist working in the BFSI sector, the contrast between ACID and BASE database architectures is more than just a technical consideration — it often reflects the tradeoff between consistency and scalability, and between reliability and flexibility. ACID (Atomicity, Consistency, Isolation, Durability) is a foundational principle for relational databases, and it's particularly crucial in the financial domain where transactional integrity is paramount. For example, when updating balances across savings and credit accounts or processing trades, any failure in transaction guarantees can lead to severe legal and financial implications.

Necessity of BASE

ACID-compliant databases such as PostgreSQL, Oracle, or SQL Server ensure that operations are all-or-nothing and immediately consistent, making them the default choice in core banking systems or credit risk models where data reliability is non-negotiable. However, as financial platforms modernize and scale—especially with the rise of mobile banking, real-time analytics, and personalization—there’s an increasing need for systems that can handle massive volumes of distributed data with lower latency. This is where BASE (Basically Available, Soft state, Eventually consistent) systems like Cassandra, DynamoDB, or MongoDB come into play.

Done

Hybrid Use Cases in BFSI

While BASE sacrifices immediate consistency, it enables high availability and horizontal scalability, which are beneficial for real-time fraud detection systems or customer behavior analytics. For instance, in one project I worked on involving real-time credit card transaction scoring, we used a hybrid approach: incoming data was captured through an eventually consistent NoSQL stream for speed, but core decisioning and ledger updates happened via ACID-compliant stores to ensure precision and traceability.

Ultimately, ACID and BASE are not mutually exclusive but complementary. In the BFSI sector, a layered architecture is increasingly common — using BASE systems for ingestion, caching, and real-time responsiveness, while relying on ACID systems for final reconciliation and compliance. The key is understanding the risk tolerance and data criticality for each component of the system. With careful design, both paradigms can coexist and provide the best of both worlds.

Enter optional comments below

Submit

The total number of points for this assignment is 10. Your peers will give you a grade from 0 - 6. You will get 2 for each peer assignment you assess. You need to grade a minimum of 2 peer assignments. You can grade up to 5 peer assignments if you like but grading extra assignment will not increase your score.